

Interlock

Arun Brahma

September 13, 2026

Interlock

The authorization layer that decides whether an AI agent's payment is allowed to settle.

Arun Brahma · contact@arunbrahma.com

[Live demo](#) · [Source](#)

Agents can already pay

x402, AP2 and stablecoin rails solved settlement. Agent wallets shipped at Coinbase, Robinhood and Binance in the last few months. An agent holding a key and moving value is no longer a research question.

Settlement is done. That is not where the remaining problem is.

Nobody solved authority

The unsolved question is not *can* the agent pay. It is **who decides whether it should**, and who can defend that decision afterwards.

May 2026 - roughly \$200K. A Morse-code prompt injection made Grok emit a transfer instruction. An automated trading agent executed it on-chain before most of it was recovered. (*Slow-Mist*)

February 2026 - \$1.78M. An oracle bug in AI-assisted contract code triggered a bad debt event on Moonwell. AI, human reviewers and a governance vote all signed off. Nothing in the review chain caught it. (*rekt.news*)

Both are the same failure: an action was taken, and no layer in the stack was responsible for deciding whether it was permitted.

Static controls cannot see this

Today's controls are wallet-level policy: allowlists, per-transaction caps, velocity limits. They are necessary and they are not sufficient.

Consider a payment where:

- the recipient **is** on the allowlist
- the amount **is** under the cap
- the budget **is** not exhausted
- the chain and asset **are** correct

Every static rule passes. The payment is still theft, because a poisoned web page the agent read in the middle redirected it. Rules that only look at the transaction cannot see what produced it.

What Interlock does

Three layers, cheapest first, short-circuiting.

01 - Deterministic. Counterparty allowlist, per-payment cap, cumulative budget, currency and chain. Microseconds.

02 - Provenance. Taint tracking applied to payments. The agent's instruction chain is typed: every step carries a trust level - `principal` (the human), `tool` (trusted structured output), `content` (untrusted fetched content), `agent`. Microseconds.

03 - Semantic. A model judges intent-to-payment consistency. Optional, off by default, and only sees what the cheap layers could not settle.

Provenance is the idea

The question the middle layer asks is not “is this payment sensible”. It is **did anything untrusted get between the plan and the payment**. That is structural, so it is answered structurally.

Three rules:

- `override_language` - untrusted content carries imperative text aimed at the agent: *ignore previous, new instructions, instead send*.
- `plan_divergence` - the payment's recipient or amount differs from the last trusted plan, **and** untrusted content intervened between that plan and the payment.
- `concealment` - untrusted content instructs the agent not to disclose a step.

A principal changing their mind is not an attack. An agent revising its own plan is not an attack. Untrusted content redirecting money is.

Signed decisions

Every decision is signed with Ed25519 - the same curve Solana uses for account keys - over a canonical payload of the verdict and every check that produced it.

That turns “who approved this payment” from an argument into a lookup. It is the artifact the Moonwell review chain did not have.

See it stop an attack

Real output from `python -m demo.run_attack`, unedited:

```
INJECTED -> DENIED

instruction chain:
  [0] (principal) user
  [1] (tool) search-tool
  [2] (content) web-page(fare-listing) <- untrusted
  [3] (agent) agent

checks:
  [pass] counterparty: merchant.agency.example is on the mandate allowlist
  [pass] amount:      595 USDC against a per-payment cap of 600
  [pass] budget:      595 of 1200 USDC cumulative
  [pass] rail:        USDC on solana against mandate USDC on solana
  [FAIL] override_language: "ignore previous", "instead send", "system:"
    in web-page(fare-listing)
  [FAIL] plan_divergence: payment to merchant.agency.example for 595
    diverges from the trusted plan (merchant.airline.example, 540)
    after untrusted content from web-page(fare-listing)
  [FAIL] concealment:    "do not mention" in web-page(fare-listing)

ed25519 signature 7359a1b4ff8d3d38f9d83d38ccc5c5d3...
```

Four static checks pass. Three provenance checks fail. The payment is denied and the reason is legible, specific and signed.

The same two scenarios run in the browser at the live demo.

How it integrates

One MCP tool call before the agent signs:

```
authorize_payment(intent, mandate, payment, instruction_chain) -> signed decision
```

Or one HTTP POST. Interlock never holds keys and never touches settlement, so it stays neutral across rails - which is also why an agent platform can adopt it without routing its risk decisions through a competitor.

Building on Solana first (x402 is live there), Base second. Authorization is a policy layer above settlement, so the long-term position is chain-agnostic.

Priced per decision

Per authorization decision and on risk reduction. **Never a percentage of payment value.**

That model only works if a decision is nearly free to produce. It is the reason the layers are ordered the way they are: an authorization layer that costs a model call per payment cannot sit inline in a checkout, and cannot be sold per decision.

It also means revenue does not wait for agent payment volume to become real.

What it does not catch yet

Provenance analysis catches what is structurally visible. It will **not** catch a payment that is structurally clean but semantically wrong - the right merchant, the right amount, for the wrong thing.

That is layer 03, and it is the next thing to build. Saying so now is cheaper than being found out later.

Why me

Production multi-agent systems, on the buy side of exactly this problem.

- **Walmart Global Tech** (Senior MLE) - built the agentic shopping assistant: a multi-agent architecture on MCP orchestrating Text-to-SQL, graph and RAG sub-agents. +37% conversion across pilot categories through intent routing and context retrieval. The recurring blocker was always the same: the moment the agent can transact, you need a defensible reason why an action was allowed.

- **Citi** - enterprise Text-to-SQL analytics on a fine-tuned Llama over financial data, under financial data governance. +65% analyst productivity.
- **Carelon** - hybrid RAG (Qdrant + BM25) over health insurance policy documents. +23% customer satisfaction.
- **Open source** - ~1,050 GitHub stars across four projects: vision-parse (481), finetuned-qlora-falcon7b-medical (261), pdf-to-markdown (204), purr (105).

Intent routing and instruction provenance are ML problems, not crypto problems. That is the ground I already stand on.

The ask

R[3]sidency x Construct. Twelve weeks in London to put this in front of the teams already shipping agent wallets and agent commerce - the buyers who need policy controls before their customers get exploited, not after.

Stage: working proof of concept, publicly runnable. Solo founder. Not yet incorporated. No users yet.

[Live demo](#) · [Source](#) · contact@arunbrahma.com